

Determining the Minimum-Area Encasing Rectangle for an Arbitrary Closed Curve

H. Freeman and R. Shapira
New York University

This paper describes a method for finding the rectangle of minimum area in which a given arbitrary plane curve can be contained. The method is of interest in certain packing and optimum layout problems. It consists of first determining the minimal-perimeter convex polygon that encloses the given curve and then selecting the rectangle of minimum area capable of containing this polygon. Three theorems are introduced to show that one side of the minimum-area rectangle must be collinear with an edge of the enclosed polygon and that the minimum-area encasing rectangle for the convex polygon is also the minimum-area rectangle for the curve.

Key Words and Phrases: enclosed curve, optimum layout, optimum packing, minimum-area encasing rectangle

CR Categories: 5.19, 5.49

Copyright © 1975, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

The work reported here was supported by the Air Force Office of Scientific Research Directorate of Mathematical and Information Sciences, under Grant AFOSR-70-1854. Authors' address: Computer Research Laboratory, Division of Applied Science, New York University, 26 Stuyvesant Street, New York, NY 10003.

¹ If a side of the rectangle touches the curve at more than one point, both the first and last such points are taken as vertices of the polygon.

Introduction

We shall consider the problem of finding the rectangle of minimum area that will enclose a given simply-connected, closed curve of arbitrary irregularity. This problem occurs in a variety of industrial applications, specifically those related to optimum two-dimensional layout [1-3]. An algorithm for finding the minimum-area rectangle will be described together with a proof that the resulting rectangle is indeed minimal.

If we are given an arbitrary closed curve (say, in graphical form), we must first convert it into a form suitable for computer processing. We shall utilize the so-called chain-coding scheme [4, 5]. In this scheme an arbitrary plane curve is represented in terms of a connected sequence of short, straight-line segments by overlaying a fine-spaced square grid and then connecting in sequence the grid nodes that lie closest to the intersections of the curve with the grid. The short, straight-line segments (also called *links*) connecting successive grid nodes can assume only eight possible directions, $0^\circ, 45^\circ, 90^\circ, \dots, 315^\circ$. In the chain code, the octal digits 0 to 7 are used to represent these directions in the counter-clockwise sense beginning with the positive x direction. For a grid of assumed unit spacing, all even-valued links will thus be of unit length and all odd-valued links, of length $\sqrt{2}$.

To find the minimum-area encasing rectangle we shall proceed in two steps. First, we shall find the minimum-perimeter convex polygon that encases the given curve (i.e. its convex hull), and then we shall determine the minimum-area rectangle that encases this convex polygon. We shall show that the minimum-area rectangle for the convex hull is also the minimum-area rectangle for the given curve.

Determination of Minimum-Perimeter Convex Polygon

We begin by encasing the given curve in a rectangle with sides parallel to the x and y axes of a cartesian coordinate system. For a curve represented in chain form $A = a_1 a_2, \dots, a_n$ where the a_i are the octal-valued chain links, one computes for $j = 0, \dots, n$,

$$\begin{aligned} W_{\min} &= \min_j \sum_{i=0}^j a_{ix}, & W_{\max} &= \max_j \sum_{i=0}^j a_{ix} \\ H_{\min} &= \min_j \sum_{i=0}^j a_{iy}, & H_{\max} &= \max_j \sum_{i=0}^j a_{iy} \end{aligned} \quad (1)$$

where the a_{ix} and a_{iy} are the x and y components of the vector denoted by a_i , and $a_{0x} = a_{0y} = 0$. The rectangle formed by the lines $y = H_{\min}$, $y = H_{\max}$, $x = W_{\min}$, and $x = W_{\max}$ will just enclose the given curve. It will, of course, in general, not be of minimum area.

Next we form a convex polygon by connecting in sequence with straight-line segments the points at which the rectangle touches the curve.¹ This is illustrated in

Figure 1, where the points are labeled with the numbers of the chain links a , that terminate there. For each edge of this polygon, the maximum outward distance to the curve is computed. A method for this has been described by Ramer [6]; however, the following method is believed to be somewhat faster [7]. Consider, say, the edge 2-30 of Figure 1, oriented at an angle θ with respect to the positive x axis. To find the maximum outward distance to the curve segment between points 2 and 30, the chain links representing the curve segment are projected on to an axis v perpendicular to the edge, as illustrated in Figure 2.

The respective v -axis projections, a_{iv} , are:

link	a_{iv}	link	a_{iv}
0	$-b$	4	b
1	d	5	$-d$
2	a	6	$-a$
3	c	7	$-c$

where

$$\begin{aligned} a &= \cos \theta & c &= \cos \theta + \sin \theta \\ b &= \sin \theta & d &= \cos \theta - \sin \theta \end{aligned} \quad (2)$$

Now

$$V_{\max} = \max_j \sum_{i=r}^j a_{iv} \quad j = r, r+1, \dots, s, \quad (3)$$

where $a_r \dots a_s$ is the chain segment lying between points 2 and 30 in Figure 1. The value of j at which $V_{\max}$ occurs identifies the point on the chain segment that is most distant (in an outward sense) from the line segment 2-30. Similar points can be found for the segments 38-63, 64-115, and 115-134. These new points (labeled 17, 59, 81, and 125, respectively) are added to the set of original vertices to form the new polygon 2-17-30-38-59-63-64-81-115-125-134-2, as shown in Figure 3.

The process of finding the points at which the chain-encoded curve is maximally distant in an outward sense from the associated polygon edge and using these points for further segmentation is repeated until the curve is completely encased. For the curve of Figure 1, this occurs in one more iteration, during which the vertices 15, 80, and 110 are added, as shown in Figure 3. Since the chain representation of a curve is polygonal in form, the convex hull can always be found in this manner in a finite number of iterations.

If approximation to a specified tolerance is sufficient, the process may be discontinued when for a particular edge a value of $V_{\max}$ is obtained that lies within the specified tolerance. The chain segment corresponding to this edge is then not divided any further. The segmentation process continues for the remaining edges until $V_{\max}$ lies within tolerance for all edges. In practice the iterative process may, of course, be stopped when for an edge the value of $V_{\max}$, though not zero, falls below the resolution limit of the chain-coding grid.

In the search for the maxima of the sums $\sum a_{iv}$ there is a potential round-off problem since the a_{iv} are

Fig. 1. Arbitrary closed curve, chain-encoded and encased in rectangle with sides parallel to x and y axes.

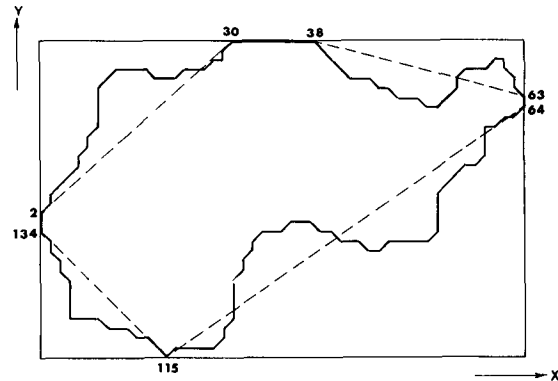


Fig. 2. Projection of chain links on to axis v , perpendicular to edge 2-30.

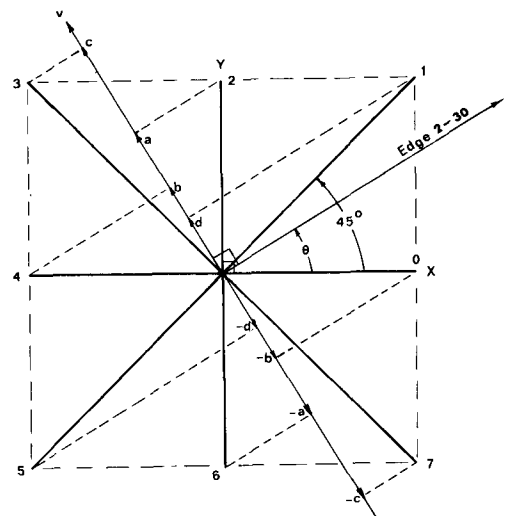
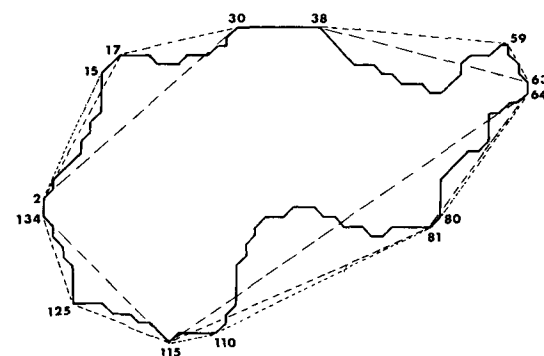


Fig. 3. Illustration of forming encasing convex polygon by successive vertex generation.



not integers and a large number of terms a_{iv} may have to be summed. The difficulty can be avoided by keeping the sums in the form of four-tuples (z_1, z_2, z_3, z_4) , where the z_i are integer sums on the link projections a, b, c , and d , respectively, and retaining only those four-tuples that are not "covered" by others. Thus, for example, the four-tuple $(7, 3, 5, 4)$ "covers" the four-tuple $(6, 1, 5, 0)$

Fig. 4. Formation of rectangle R' by counterclockwise edge rotation.

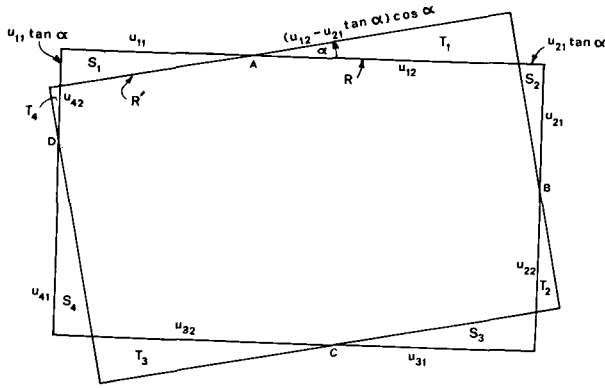


Fig. 5. Formation of rectangle R' by clockwise edge rotation.

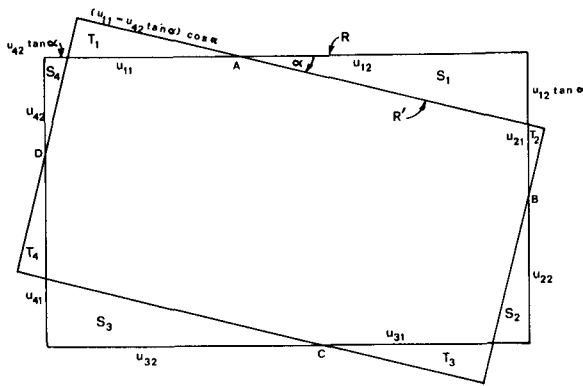
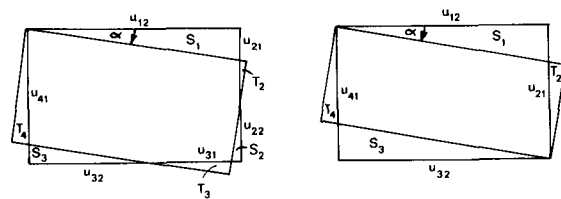


Fig. 6. Two degenerate cases involving overlapping points.



since the former will always be of larger value, regardless of the actual values of the positive constants a , b , c , and d . The set of sums $\sum a_{iv}$ that are not covered by others forms a small subset of the set of $\sum a_{iv}$; the maximum sum term is then easily determined by direct evaluation [7].

Determination of Minimum-Area Rectangle

To obtain the minimum-area encasing rectangle for a given convex polygon, we shall make use of three theorems as follows:

THEOREM 1. *Given a rectangle with four points arbitrarily chosen such that no edge contains more than one point, there exists another rectangle such that each edge*

contains one and only one of these points and the area of the rectangle is less than that of the given rectangle.

PROOF. Select a rectangle R with one point on each of its sides such that the sides u_i are each divided into two segments, $u_i = u_{i1} + u_{i2}$, $i = 1, \dots, 4$. We now form another rectangle R' with sides passing through the same points and such that each side of R' makes an angle α in the counterclockwise sense with the corresponding side of R , as illustrated in Figure 4, where the four points are marked A , B , C , and D .

Let ΔA be the amount by which the area of R exceeds that of R' . Using the notation of Figure 4, we can write:

$$\Delta A = \sum_{i=1}^4 S_i - \sum_{i=1}^4 T_i \quad (4)$$

where

$$S_i = \frac{1}{2} u_{i1}^2 \tan \alpha \quad (5)$$

$$T_i = \frac{1}{2} (u_{i2} - u_{i+1,1} \tan \alpha)^2 \sin \alpha \cos \alpha \quad (6)$$

Upon substituting and simplifying,

$$\Delta A = \sin^2 \alpha \sum_{i=1}^4 u_{i2} u_{i+1,1} + \frac{1}{2} \sin \alpha \cos \alpha \left[\sum_{i=1}^4 u_{i1}^2 - \sum_{i=1}^4 u_{i2}^2 \right] \quad (7)$$

$$= K_1 + K_2 \left[\sum_{i=1}^4 u_{i1}^2 - \sum_{i=1}^4 u_{i2}^2 \right] \quad (8)$$

where K_1 and K_2 are both positive.

We now examine the case where R' is obtained by rotating its sides by an amount α in the clockwise sense, as shown in Figure 5. Now,

$$S_i = \frac{1}{2} u_{i2}^2 \tan \alpha \quad (9)$$

$$T_i = \frac{1}{2} (u_{i1} - u_{i-1,2} \tan \alpha)^2 \sin \alpha \cos \alpha \quad (10)$$

$$\Delta A = \sum_{i=1}^4 S_i - \sum_{i=1}^4 T_i \quad (11)$$

$$= \sin^2 \alpha \sum_{i=1}^4 u_{i1} u_{i-1,2} + \frac{1}{2} \sin \alpha \cos \alpha \left[\sum_{i=1}^4 u_{i2}^2 - \sum_{i=1}^4 u_{i1}^2 \right] \quad (12)$$

$$= K_1 - K_2 \left[\sum_{i=1}^4 u_{i1}^2 - \sum_{i=1}^4 u_{i2}^2 \right] \quad (13)$$

ΔA is the amount by which the area of R' is less than that of R . It is apparent from (8) and (13) that ΔA is positive in at least one of the two cases if not in both.

By letting $u_{11} = u_{42} = 0$, or $u_{11} = u_{42} = u_{22} = u_{31} = 0$, the proof also covers the two special cases in which points overlap (see Figure 6).

THEOREM 2. *The rectangle of minimum area enclosing a convex polygon has a side collinear with one of the edges of the polygon.*

PROOF. Let us assume that the theorem is false. Since the polygon is convex, only one of its vertices can be on any one side of the rectangle. Since no polygon edge lies on a rectangle side, we can construct another rectangle which will also enclose the polygon, and, by

Theorem 1, will be of lesser area. Hence the given rectangle could not have been of minimum area.

THEOREM 3. *The minimum-area rectangle encasing the convex hull of a simple, closed, chain-coded curve is one and the same as the minimum-area rectangle encasing the curve.*

PROOF. A chain-coded curve is a polygon. Denote it by P . A convex polygon encasing P is the convex hull, C , of P if and only if the set of vertices of the convex polygon is a subset of the vertices of P . The minimum-area rectangle encasing P thus encases all vertices of C , and, because of its own convexity, encases C . Therefore, the minimum-area rectangle encasing P cannot be smaller than the minimum-area rectangle encasing C . Obviously it is not greater since the minimum-area rectangle encasing C also encases P .

To find the minimum-area rectangle for a given chain-coded curve, it is thus only necessary to find the convex hull and then to test the set of rectangles having one side collinear with the polygon's edges for the rectangle with least area.

For illustration consider the 134-link chain-coded curve shown in Figure 1:

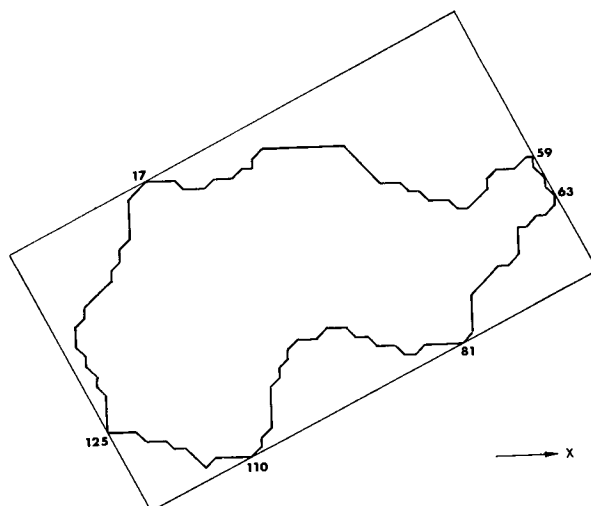
```
22121 11212 12222 11000 70010 01021 00000 00077 77007
07007 01121 00106 76765 45466 65455 56666 54444 54344
34344 54456 56566 66565 44445 33434 43444 22223 2323
```

When the foregoing method is applied to this chain, the convex hull 134-2-15-17-30-38-59-63-64-80-81-110-115-125-134 is obtained, as shown in Figure 3. Evaluation of the 14 rectangles that can be constructed by using as a base the 14 edges of the convex hull in succession shows that the minimum-area rectangle is the one based on either the edge 59-63 or the edge 81-110—both of which happen to yield the same result since they are perpendicular to each other (a coincidence). The area for the minimum-area rectangle is 1589 square units, compared to 1592 square units for the second-best rectangle (based on the edge 17-30) and 1650 square units for the original x-y axis rectangle of Figure 1; see Figure 7.

It is worthwhile to note that if it appears that the convex hull will have a large number of edges (say, more than 24), the test evaluation of the rectangles may be made after, say, the third iteration even though the polygon being tested is not the convex hull. For the rectangle base lines, the lines are drawn parallel to the polygon edges and tangent to the chain-coded curve. All edges but the three or four yielding the least-area rectangles are then not considered any further. The iterations then continue with the remaining edges until the corresponding part of the convex hull is found, at which point the final rectangle evaluation is carried out. The actual chain-coded curve (rather than the polygon) is, of course, then used to determine the rectangle corresponding to each edge of this part of the convex hull.

Since it is assumed that the curve is given in chain-coded form, the convex hull is likely to contain edges whose slopes correspond to some or all of the eight directions of the chain code ($m45^\circ$, $m = 0, 1, \dots, 7$).

Fig. 7. The minimum-area rectangle for the illustration of Figure 1.



Clearly, in such a case only the edges corresponding to the slopes 0° and 45° need then be evaluated, the others yielding redundant rectangles.

Outline of Algorithm

The following is a description of the algorithm in outline form:

- Step 1. The given closed curve is to be encoded into a chain $a_1 a_2 \dots a_n$ with the direction of encoding such that the interior lies toward the right.
- Step 2. Determine $W_{\min}$, $W_{\max}$, $H_{\min}$, and $H_{\max}$ using eq. (1). Find the coordinate pairs of all the chain points lying on these lines. If more than two chain points lie on one line, ignore all but the first and last. Label the selected chain point with the number of the chain link directed toward it and enter it in an ordered vertex list. (Thus V_i is the vertex toward which the link a_i is directed.)
- Step 3. Form a closed polygon from the ordered set of vertices. For each chain section lying between two successive vertices, compute the coordinates of the chain point(s)² maximally distant toward the left from the line segment connecting the two vertices. (The computational method for this was described in connection with eq. (3).)
- Step 4. If the maximum distance toward the left is greater than zero, the corresponding chain point(s) are labeled as in Step 2 and entered in the ordered vertex list. If there are more than two such chain points for one chain section, only the first and last are entered in the vertex list.
- Step 5. Repeat Steps 3 and 4 for all pairs of successive vertices in the vertex list except those already processed. When the maximum distance toward the left

² Two or more points may have the same maximum distance value.

is equal to zero for all pairs of successive vertices in the vertex list, go to Step 6.

Step 6. Select a pair of successive vertices V_i, V_j ($j > i$ except when end point of chain lies between the two vertices). Set up an x', y' coordinate system with origin at V_i and positive x' axis directed toward V_j . Compute the x', y' coordinates for all the vertices in the vertex list.

Step 7. Search the vertex list for the values of $\max x'$, $\min x'$, and $\min y'$. The area of the encasing rectangle is given by $(\max x' - \min x') \times (-\min y')$.

Step 8. Repeat Steps 6 and 7 until the areas of the rectangles corresponding to all pairs of successive vertices have been computed. The rectangle of minimum area is the desired minimum-area encasing rectangle for the given chain-coded curve.

Received May 1974; revised December 1974

References

1. Haims, M.J., and Freeman, H. A multistage solution of the template-layout problem. *IEEE Trans. Syst. Science and Cybernetics* SSC-6, 2 (Apr. 1970), 145-151.
2. Adamowicz, M., and Albano, A. A two-stage solution of the cutting stock problem. *Information Processing 71 Proc. IFIP Cong. 71*, North Holland Pub. Co., Amsterdam, 1972, pp. 1086-1091.
3. Eastman, C.M. Heuristic algorithms for automated space planning. *Proc. 2nd Int'l. Conf. on Artif. Intell.*, British Computer Society, 29 Portland Place, London, 1971, pp. 27-39.
4. Freeman, H. On the encoding of arbitrary geometric configurations, *IRE Trans. EC-10*, 2 (June 1961), 260-268.
5. Freeman, H. Boundary encoding and processing. In *Picture Processing and Psychopictorics*, B. Lipkin and A. Rosenfeld (Eds). Academic Press, New York, 1970.
6. Ramer, E.U. An iterative procedure for the polygonal approximation of plane curves. *Computer Graphics and Image Processing* 1, 3 (Nov. 1972), 244-256.
7. Freeman, H. Computer processing of line-drawing images. *Computing Surveys* 6, 1 (Mar. 1974), 57-97.

Short Communication Programming Techniques

A Note on Hash Linking

Daniel G. Bobrow
Xerox Palo Alto Research Center

Key Words and Phrases: hash search, address space extension, implicit pointers, monitor conditions

CR Categories: 3.7, 3.73, 4.1

In current machine designs, a machine address gives the user direct access to a single piece of information, namely the contents of that machine word. This note is based on the observation that it is often useful to associate additional information, with some (relatively few) address locations determined at run time, without the necessity of preallocating the storage at all possible such addresses. That is, it can be useful to have an effective extra bit, field, or address in some words without every word having to contain a bit (or bits) to mark this as a special case. The key idea is that this extra associated information can be found by a table search. Although it could be found by any search technique (e.g. linear, binary sorted, etc.), we suggest that an appropriate low overhead mechanism is to use hash search on a table in which the key is the address of the cell to be augmented.

Hash searching (see Knuth, [2]) is a technique in which a key is mapped into a unique address associated with that key. Most applications of this technique are for insertion and fast retrieval of data records containing key fields. In our use of hash search, the key field is the virtual address of a machine cell with which we wish to associate additional information. Since we *link* an address to auxiliary data not contained in that cell, we call this process *hash linking*. (A hash link function is one which maps any machine virtual address into another unique address where additional information can be stored.) This note describes several nonobvious applications of this technique.

Marked Linking

In marked linking, the contents of a cell to which a

Copyright © 1975, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

Author's address: Xerox Palo Alto Research Center, 3333 Coyote Hill Road, Palo Alto, CA 94304.